TestNG

TestNG是一个java中的开源自动化测试框架，其灵感来自JUnit和NUnit，TestNG还涵盖了JUnit4整个核心的功能，但引入了一些新的功能，使其功能更强大，使用更方便。

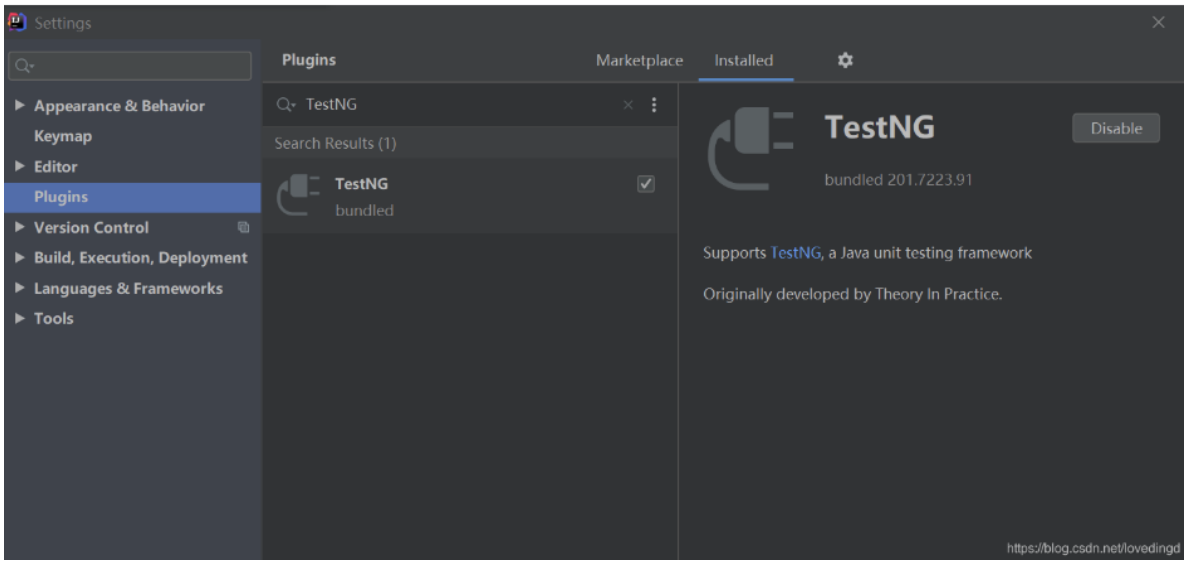
优势：支持依赖测试方法，并行测试，负载测试，局部故障，灵活的插件API，支持多线程测试。详细使用说明请参考官方链接：TestNG官方文档。

一，TestNG使用流程

1. TestNG安装

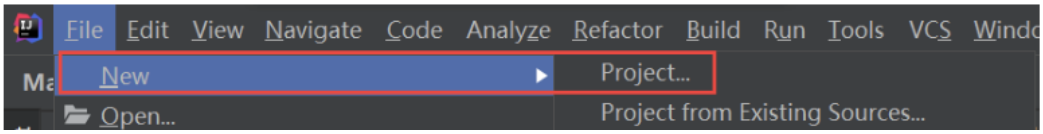
以IDEA+Maven为例介绍TestNG

IntelliJ IDEA中默认集成了TestNG，点击File->Settings，如下图：

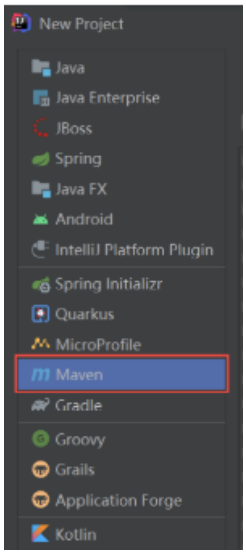


1.2 创建Maven项目

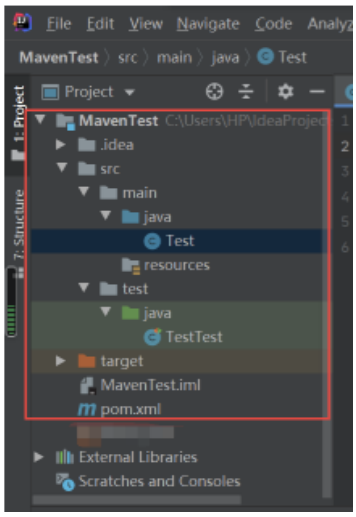
点击File->new-Project，如图



创建基于Maven的项目



创建名字为MavenTest的项目，创建完成后如下图



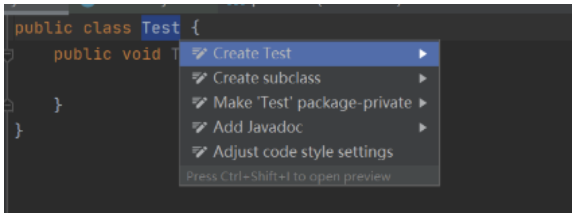
1.3 Maven配置

在工程的pom.xml中需要添加如下依赖：

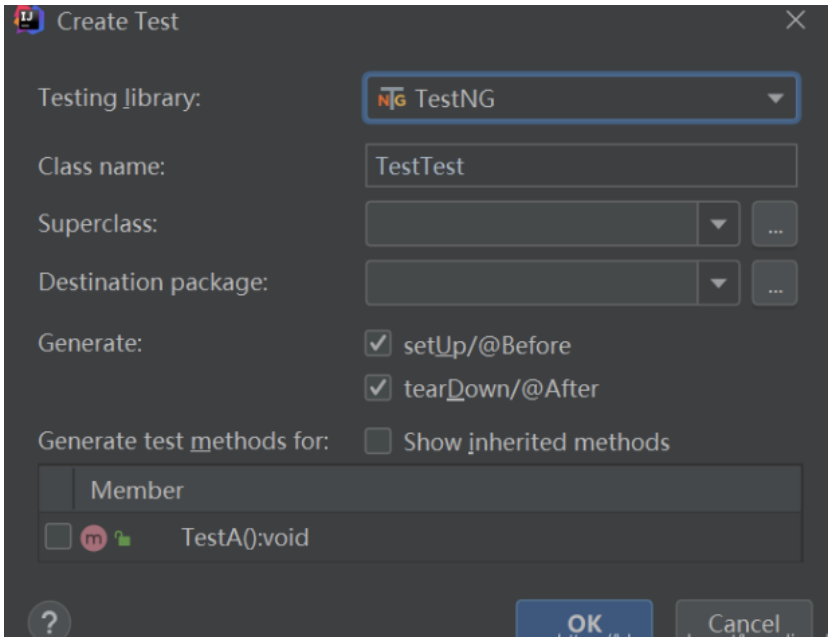
```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>MavenTest</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>6.10</version>
        </dependency>
    </dependencies>
</project>
```

1.4 项目TestNG测试类

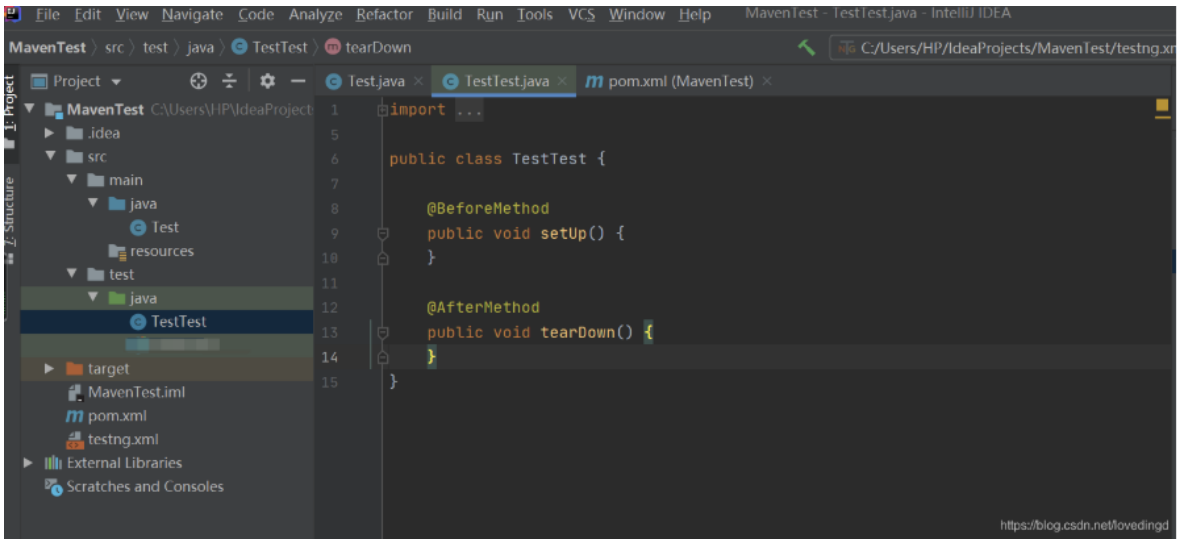
点击类名Test，按alt+entet键，创建单元测试类



点击Create Test

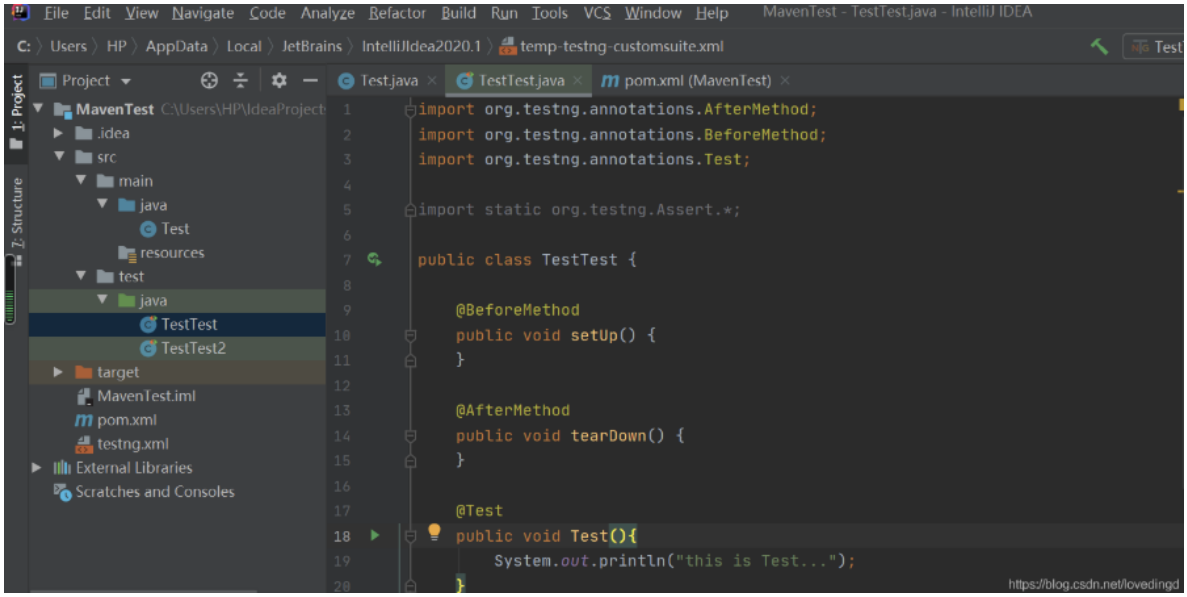


生成测试类

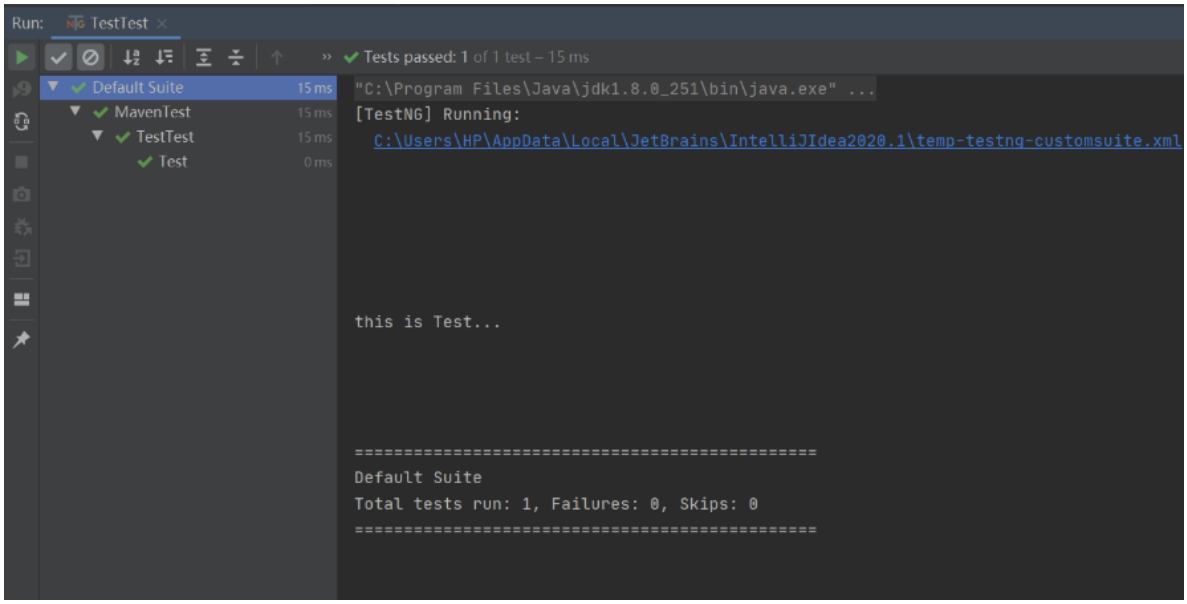


1.5 运行TestNG

添加一个测试方法，如下



运行结果如下：



二，TestNG常用注解

有必要介绍一下TestNG注解的生命周期，先看一下官网支持的注解有：

注解	描述
@BeforeSuite	在该套件的所有测试运行之前运行，仅运行一次
@AfterSuite	在该套件的所有测试运行之后运行，仅运行一次
@BeforeTest	注释的方法将在属于test标签内的所有类的所有测试方法运行之前运行，一个test标签内可能有多个class（类），在当前test标签内仅运行一次
@AfterTest	注释的方法将在属于test标签内的所有类的所有测试方法运行之后运行，一个test标签内可能有多个class（类），在当前test标签内仅运行一次
@BeforeClass	在调用当前类的第一个测试方法之前运行，在当前类中仅运行一次
@AfterClass	在调用当前类的第一个测试方法之后运行，在当前类中仅运行一次
@BeforeGroups	配置方法将在组列表运行之前运行。 此方法保证在调用属于这些组中的任何第一个测试方法之前不久运行
@AfterGroups	此配置方法将在组列表运行之后运行。该方法保证在调用属于这些组的任何最后一个测试方法之后不久运行
@BeforeMethod	注释方法将在当前类中的每个测试方法之前运行
@AfterMethod	注释方法将在当前类中的每个测试方法之后运行
@Parameters	描述如何将参数传递给@Test方法
@DataProvider	标记一种方法来提供测试方法的数据。 注释方法必须返回一个Object [] []，其中每个Object []可以被分配给测试方法的参数列表。 要从该DataProvider接收数据的@Test方法需要使用与此注释名称相等的dataProvider名称
@Listeners	定义测试类上的侦听器
@Factory	将一个方法标记为工厂，返回TestNG将被用作测试类的对象。 该方法必须返回Object []
@Test	将类或方法标记为测试的一部分，此标记若放在类上，则该类所有公共方法都将被作为测试方法

如上列表中的@Factory、@Linsteners这两个是不常用的；
前十个注解看起来不太容易区分，顺序不太容易看明白，以如下范例做简单说明，代码

```
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterGroups;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.AfterSuite;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeGroups;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.BeforeSuite;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;
```

```java
public class NewTest {

  @Test(groups="group1")
  public void test1() {
      System.out.println("test1 from group1");
      Assert.assertTrue(true);
  }

  @Test(groups="group1")
  public void test11() {
      System.out.println("test11 from group1");
      Assert.assertTrue(true);
  }

  @Test(groups="group2")
  public void test2()
  {
      System.out.println("test2 from group2");
      Assert.assertTrue(true);
  }

  @BeforeTest
  public void beforeTest()
  {
      System.out.println("beforeTest");
  }

  @AfterTest
  public void afterTest()
  {
      System.out.println("afterTest");
  }

  @BeforeClass
  public void beforeClass()
  {
      System.out.println("beforeClass");
  }

  @AfterClass
  public void afterClass()
  {
      System.out.println("afterClass");
  }

  @BeforeSuite
  public void beforeSuite()
  {
      System.out.println("beforeSuite");
  }

  @AfterSuite
  public void afterSuite()
  {
      System.out.println("afterSuite");
  }
```

```
  //只对group1有效，即test1和test11
  @BeforeGroups(groups="group1")
  public void beforeGroups()
  {
      System.out.println("beforeGroups");
  }

  //只对group1有效，即test1和test11
  @AfterGroups(groups="group1")
  public void afterGroups()
  {
      System.out.println("afterGroups");
  }

  @BeforeMethod
  public void beforeMethod()
  {
      System.out.println("beforeMethod");
  }

  @AfterMethod
  public void afterMethod()
  {
      System.out.println("afterMethod");
  }
}
```

运行结果如下：

```
beforeSuite
beforeTest
beforeClass
beforeGroups
beforeMethod
test1 from group1
afterMethod
beforeMethod
test11 from group1
afterMethod
afterGroups
beforeMethod
test2 from group2
afterMethod
afterClass
afterTest
PASSED: test1
PASSED: test11
PASSED: test2

===============================================
    Default test
    Tests run: 3, Failures: 0, Skips: 0
===============================================

afterSuite
```

由此可见，testng运行时，顺序是这样的：

1. **全局级（Suite）**：`@BeforeSuite` / `@AfterSuite` 整个测试套件仅执行 1 次（不管有多少测试类）；
2. **测试集级（Test）**：`@BeforeTest` / `@AfterTest` 对应 `testng.xml` 中 `<test>` 标签，一个 `<test>` 标签执行 1 次；
3. **类级（Class）**：`@BeforeClass` / `@AfterClass` 每个测试类执行 1 次（在类中所有用例执行前后）；
4. **分组级（Groups）**：`@BeforeGroups` / `@AfterGroups` 仅对指定分组生效，且**该分组第一个用例执行前触发 BeforeGroups，最后一个用例执行后触发 AfterGroups**；
5. **方法级（Method）**：`@BeforeMethod` / `@AfterMethod` 每个测试用例（`@Test` 方法）执行前后各触发 1 次（不管属于哪个分组）。

```
@BeforeSuite->@BeforeTest->@BeforeClass->{@BeforeGroups[@BeforeMethod->@Test1->@AfterMethod，@BeforeMethod->@Test2->@AfterMethod, ...]@AfterGroups, @BeforeGroups[@BeforeMethod->@Test3->@AfterMethod，@BeforeMethod->@Test4->@AfterMethod, ...]@AfterGroups, ...}->@AfterClass->@AfterTest->@AfterSuite
```

其中{}内将@BeforeGroups@AfterGroups将@Test分组，每个分组内包含多个@Test。

三，xml方式运行

TestNG也可以以xml的方式运行

```
<suite>  套件，根标签，通常由几个<test组成>
  属性：
    name              套件的名称，必须属性；
    verbose           运行的级别或详细程度；
    parallel          是否运行多线程来运行这个套件；
    thread-count      如果启用多线程，用于指定开户的线程数；
    annotations       在测试中使用的注释类型；
    time-out          在本测试中的所有测试方法上使用的默认超时时间；
<test>      测试用例，name为必须属性；
<classes>   用例中包含的类，子标签为<class name="className">；
<class>     测试类，其中属性name为必须属性；；
<packages> 用例中包含的包，包中所有的方法都会执行，子标签为<package name="packageName">；
<package>   测试包，name为必须属性；
<methods>   指定测试类中包含或排除的方法，子类为<include>,<exclude>；
<include>   指定需要测试的方法，name为必须属性；
<exclude>   指定类中不需要测试的方法，name为必须属性；
<groups>    指定测试用例中要运行或排除运行的分组，子标签为<run>,<run>下包含<include>,
<exclude>标签，<include>,<exclude>的name指定运行、不运行的分组；
```

在项目下新建一个testng.xml文件，模板如下：

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >

<suite name="Suite1" verbose="1" >
  <test name="Nopackage" >
    <classes>
       <class name="NoPackageTest" />
    </classes>
  </test>

  <test name="Regression1">
    <classes>
```

```
      <class name="test.sample.ParameterSample"/>
      <class name="test.sample.ParameterTest"/>
    </classes>
  </test>
</suite>
```

3.1 鼠标右击testng.xml运行

TestTest3的代码如下：

```
import org.testng.Assert;
import org.testng.annotations.Test;
import org.testng.annotations.*;


public class TestTest3 {


    @Test(groups = "group1")
    public void testC03() {
        System.out.println("testC03");
        Assert.assertTrue(true);
    }

    @Test(groups = "group1")
    public void testC04() {
        System.out.println("testC04");
        Assert.assertTrue(true);
    }

    @Test(groups = "group2")
    public void testC05() {
        System.out.println("testC05");
        Assert.assertTrue(true);
    }

    @BeforeTest
    public void beforeTest() {
        System.out.println("beforeTest");
    }

    @AfterTest
    public void afterTest() {
        System.out.println("afterTest");
    }

    @BeforeClass
    public void beforeClass() {
        System.out.println("beforeClass");
    }

    @AfterClass
    public void afterClass() {
        System.out.println("afterClass");
    }

    @BeforeSuite
```

```
    public void beforeSuite() {
        System.out.println("beforeSuite");
    }

    @AfterSuite
    public void afterSuite() {
        System.out.println("afterSuite");
    }

    @BeforeGroups(groups = {"group1", "group2"})
    public void beforeGroups() {
        System.out.println("beforeGroups");
    }

    @AfterGroups(groups = {"group1", "group2"})
    public void afterGroups() {
        System.out.println("afterGroups");
    }


    @BeforeMethod
    public void beforeMethod() {
        System.out.println("beforeMethod");
    }

    @AfterMethod
    public void afterMethod() {
        System.out.println("afterMethod");
    }
}
```

TestTest4的代码如下：

```
import org.testng.Assert;
import org.testng.annotations.Test;
import org.testng.annotations.*;


public class TestTest4 {


    @Test(groups = "group1")
    public void testD03() {
        System.out.println("testD03");
        Assert.assertTrue(true);
    }

    @Test(groups = "group1")
    public void testD04() {
        System.out.println("testD04");
        Assert.assertTrue(true);
    }

    @Test(groups = "group2")
    public void testD05() {
        System.out.println("testD05");
        Assert.assertTrue(true);
```

```
    }

    @BeforeTest
    public void beforeTest() {
        System.out.println("beforeTest");
    }

    @AfterTest
    public void afterTest() {
        System.out.println("afterTest");
    }

    @BeforeClass
    public void beforeClass() {
        System.out.println("beforeClass");
    }

    @AfterClass
    public void afterClass() {
        System.out.println("afterClass");
    }

    @BeforeGroups(groups = "group1")
    public void beforeGroups() {
        System.out.println("beforeGroups");
    }

    @AfterGroups(groups = "group1")
    public void afterGroups() {
        System.out.println("afterGroups");
    }

    @BeforeMethod
    public void beforeMethod() {
        System.out.println("beforeMethod");
    }

    @AfterMethod
    public void afterMethod() {
        System.out.println("afterMethod");
    }
}
```

修改testng.xml的内容如下：

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd" >

<suite name="Suite" parallel="none">
    <test name="Test">
<!--        <groups>-->
<!--            <run>-->
<!--                <include name="group1"/>-->
<!--            </run>-->
<!--        </groups>-->

        <classes>
            <class name="TestTest3"/>
```

```
        </classes>

    </test> <!-- Test -->

    <test name="Test2">
        <classes>
            <class name="TestTest4"/>
        </classes>
    </test> <!-- Test -->

</suite> <!-- Suite -->
```

鼠标右键testng.xml文件，运行结果如下：

```
[TestNG] Running:
  C:\Users\HP\IdeaProjects\MavenTest\testng.xml

beforeSuite

beforeTest

beforeClass

beforeGroups

beforeMethod

testC03

afterMethod

beforeMethod

testC04

afterMethod

afterGroups

beforeGroups

beforeMethod

testC05

afterMethod

afterGroups

afterClass

afterTest


beforeTest

beforeClass
```

```
beforeGroups

beforeMethod

testD03

afterMethod

beforeMethod

testD04

afterMethod

afterGroups

beforeMethod

testD05

afterMethod

afterClass

afterTest

afterSuite

===============================================
Suite
Total tests run: 6, Failures: 0, Skips: 0
===============================================

Process finished with exit code 0
```

3.2 使用maven运行

需要在pom文件中，指明testng.xml文件的位置。

maven使用surefire这个插件进行测试，可以执行testng或者Junit脚本。语法为

```
<suiteXmlFile>src/test/resources/testNGFilesFolder/${testNgFileName}.xml</suiteXmlFile>
```

```
<dependencies>
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-api</artifactId>
            <version>2.8</version>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
```

```
                <artifactId>maven-surefire-plugin</artifactId>
                <version>2.19</version>
                <configuration>
                    <suiteXmlFiles>
                        <suiteXmlFile>testng.xml</suiteXmlFile>//该文件位于工程根目录时，直接填写名字，其它位置要加上路径。
                    </suiteXmlFiles>
                </configuration>
            </plugin>
        </plugins>
    </build>
```

运行测试脚本

进入到项目工程的根目录，使用 mvn clean test -Dtestng.xml 命令。

四，常用的断言（assert）

```
assertEqual ([String message], expected value, actual value)        断言两个值相等。值可能是类型有 int, short, long, byte, char or java.lang.Object. 第一个参数是一个可选的字符串消息;

assertTrue([String message], boolean condition)                     断言一个条件为真;

assertFalse([String message],boolean condition)                     断言一个条件为假;

assertNotNull([String message], java.lang.Object object)            断言一个对象不为空(null);

assertNull([String message], java.lang.Object object)               断言一个对象为空(null);

assertSame([String message], java.lang.Object expected, java.lang.Object actual)
        断言两个对象引用相同的对象;

assertNotSame([String message], java.lang.Object unexpected, java.lang.Object actual)    断言两个对象不是引用同一个对象;

assertArrayEquals([String message], expectedArray, resultArray)
                        断言预期数组和结果数组相等。数组的类型可能是 int, long, short, char, byte or java.lang.Object.;
```

五，TestNG预期异常测试

预期异常测试通过在@Test注解后加入预期的Exception来进行添加，范例如下所示：

```
@Test(expectedExceptions = ArithmeticException.class)
    public void divisionWithException() {
        int i = 1 / 0;
        System.out.println("After division the value of i is :"+ i);
    }
```

运行结果如下

```
[RemoteTestNG] detected TestNG version 6.10.0
[TestNG] Running:
  C:\Users\Administrator\AppData\Local\Temp\testng-eclipse--754789457\testng-customsuite.xml

PASSED: divisionWithException

===============================================
    Default test
    Tests run: 1, Failures: 0, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 1, Failures: 0, Skips: 0
===============================================

[TestNG] Time taken by org.testng.reporters.JUnitReportReporter@55d56113: 0 ms
[TestNG] Time taken by org.testng.reporters.SuiteHTMLReporter@1e127982: 0 ms
[TestNG] Time taken by org.testng.reporters.jq.Main@6e0e048a: 32 ms
[TestNG] Time taken by [FailedReporter passed=0 failed=0 skipped=0]: 0 ms
[TestNG] Time taken by org.testng.reporters.XMLReporter@43814d18: 0 ms
[TestNG] Time taken by org.testng.reporters.EmailableReporter2@6ebc05a6: 0 ms
```

六，TestNG忽略测试

有时候我们写的用例没准备好，或者该次测试不想运行此用例，那么删掉显然不明智，那么就可以通过注解`@Test(enabled = false)`来将其忽略掉，此用例就不会运行了，如下范例：

```java
import org.testng.annotations.Test;

public class TestCase1 {

    @Test(enabled=false)
    public void TestNgLearn1() {
        System.out.println("this is TestNG test case1");
    }

    @Test
    public void TestNgLearn2() {
        System.out.println("this is TestNG test case2");
    }
}
```

运行结果

```
this is TestNG test case2
PASSED: TestNgLearn2
```

七，TestNG超时测试

“超时”表示如果单元测试花费的时间超过指定的毫秒数，那么TestNG将会中止它并将其标记为失败。此项常用于性能测试。如下为一个范例：

```
import org.testng.annotations.Test;

public class TestCase1 {

    @Test(timeOut = 5000) // time in mulliseconds
    public void testThisShouldPass() throws InterruptedException {
        Thread.sleep(4000);
    }

    @Test(timeOut = 1000)
    public void testThisShouldFail() {
        while (true){
            // do nothing
        }

    }
}
```

运行结果

```
PASSED: testThisShouldPass
FAILED: testThisShouldFail
org.testng.internal.thread.ThreadTimeoutException: Method
com.demo.test.testng.TestCase1.testThisShouldFail() didn't finish within the
time-out 1000
    at com.demo.test.testng.TestCase1.testThisShouldFail(TestCase1.java:37)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at
sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at
sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.jav
a:43)
    at java.lang.reflect.Method.invoke(Method.java:498)
    at
org.testng.internal.MethodInvocationHelper.invokeMethod(MethodInvocationHelper.j
ava:104)
    at
org.testng.internal.InvokeMethodRunnable.runOne(InvokeMethodRunnable.java:54)
    at
org.testng.internal.InvokeMethodRunnable.run(InvokeMethodRunnable.java:44)
    at java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:511)
    at java.util.concurrent.FutureTask.run(FutureTask.java:266)
    at
java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
    at
java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:748)
```

八， 分组测试

1. 分组可标注在**测试方法**（`@Test(groups="xxx")`）或**测试类**（类上加 `@Test(groups="xxx")`）上；
2. 类级分组会被所有方法继承，方法级分组与类级分组是「叠加关系」；
3. 一个用例可绑定多个分组（数组形式 `groups={"a","b"}`）；
4. `@BeforeGroups/@AfterGroups` 必须指定分组名，才会仅对该分组生效；
5. XML 中通过 `<groups>` 标签可精准控制「执行 / 排除」指定分组。

完整实操案例

1. 测试类：NewTest.class（包含所有分组场景）

```
package com.webLearn; // 请替换为你自己的包名

import org.testng.Assert;
import org.testng.annotations.*;

// 类级分组：整个类的所有方法默认属于 "smoke"（冒烟测试）分组
@Test(groups = "smoke")
public class NewTest {

    // 方法级单分组：仅属于 group1（同时继承类级的 smoke 分组）
    @Test(groups = "group1")
    public void test1() {
        System.out.println("test1 → 所属分组：group1 + smoke");
        Assert.assertTrue(true);
    }

    // 方法级单分组：仅属于 group1（继承 smoke）
    @Test(groups = "group1")
    public void test11() {
        System.out.println("test11 → 所属分组：group1 + smoke");
        Assert.assertTrue(true);
    }

    // 方法级多分组：同时属于 group2 + database（继承 smoke）
    @Test(groups = {"group2", "database"})
    public void test2() {
        System.out.println("test2 → 所属分组：group2 + database + smoke");
        Assert.assertTrue(true);
    }

    // 方法级多分组：同时属于 database + mysql（继承 smoke）
    @Test(groups = {"database", "mysql"})
    public void testDB() {
        System.out.println("testDB → 所属分组：database + mysql + smoke");
        Assert.assertTrue(true);
    }

    // 无方法级分组：仅继承类级的 smoke 分组
    @Test
    public void testSmokeOnly() {
        System.out.println("testSmokeOnly → 所属分组：smoke");
        Assert.assertTrue(true);
    }

    // 分组钩子：仅对 group1 生效（group1 第一个用例执行前触发）
```

```java
    @BeforeGroups(groups = "group1")
    public void beforeGroup1() {
        System.out.println("\n===== BeforeGroups(group1) 执行 =====");
    }

    // 分组钩子：仅对 group1 生效（group1 最后一个用例执行后触发）
    @AfterGroups(groups = "group1")
    public void afterGroup1() {
        System.out.println("===== AfterGroups(group1) 执行 =====\n");
    }

    // 分组钩子：仅对 database 生效（database 第一个用例执行前触发）
    @BeforeGroups(groups = "database")
    public void beforeDatabase() {
        System.out.println("\n===== BeforeGroups(database) 执行 =====");
    }

    // 分组钩子：仅对 database 生效（database 最后一个用例执行后触发）
    @AfterGroups(groups = "database")
    public void afterDatabase() {
        System.out.println("===== AfterGroups(database) 执行 =====\n");
    }

    // 每个测试方法执行前触发（所有分组的用例都会触发）
    @BeforeMethod
    public void beforeMethod() {
        System.out.println("→ BeforeMethod 执行");
    }

    // 每个测试方法执行后触发
    @AfterMethod
    public void afterMethod() {
        System.out.println("→ AfterMethod 执行\n");
    }
}
```

2. XML 配置：testng.xml（控制分组执行）

放在 `src/test/resources` 目录下，提供 3 种常用配置（按需注释 / 取消注释）：

```xml
<?xml version="1.0" encoding="UTF-8"?>
<suite name="GroupTest_Suite" verbose="1">
    <test name="Group_Test_Case">
        <classes>
            <!-- 指向你的测试类（包名+类名） -->
            <class name="com.webLearn.NewTest"/>
        </classes>

        <!-- 分组核心配置：<groups> 标签 -->
        <groups>
            <!-- 场景1：只执行 database 分组（最常用） -->
            <run>
                <include name="database"/>
            </run>

            <!-- 场景2：排除 group1 分组（执行除 group1 外的所有用例）
            <run>
```

```
                <exclude name="group1"/>
            </run>
            -->

            <!-- 场景3：组合执行（同时执行 group1 + mysql 分组）
            <run>
                <include name="group1"/>
                <include name="mysql"/>
            </run>
            -->
        </groups>
    </test>
</suite>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  TestNG 测试套件核心配置文件
  根标签 <suite>：定义一个测试套件（可包含多个 <test> 标签）
  - name：套件名称，用于标识当前套件（自定义）
  - parallel：是否并行执行测试用例，false 表示串行执行（默认值）
              可选值：false(串行)、methods(方法级并行)、tests(测试级并行)、classes(类级
并行)
-->
<suite name="Suite" parallel="false">

  <!--
    <test> 标签：定义一个测试集（套件下可包含多个测试集）
    - name：测试集名称，用于标识当前测试集（自定义）
  -->
  <test name="Test">

    <!--
      <groups> 标签：分组测试配置，用于筛选要执行/排除的测试分组
      核心作用：只执行指定分组的用例，或排除指定分组的用例
    -->
    <groups>
      <!--
        <include> 标签：指定要执行的分组名称（白名单）
        - name：分组名，需与测试类中 @Test(groups="xxx") 的分组名一致
        以下配置表示：执行 group1 和 group2 两个分组的所有测试用例
        注意：原代码中 </incloud> 是拼写错误，正确为 </include>
      -->
      <include name="group1"></include>
      <include name="group2"></include>

      <!-- 可选：<exclude> 标签，指定要排除的分组（黑名单）
      <exclude name="ignoreGroup"></exclude>
      -->
    </groups>

    <!--
      <classes> 标签：指定要执行的测试类列表
      作用：限定当前测试集要运行的测试类，仅执行这些类中符合分组条件的用例
    -->
    <classes>
      <!--
        <class> 标签：单个测试类的全限定名（包名 + 类名）
```

```
        第一个测试类：com.demo.test.testng.NewTest
      -->
      <class name="com.demo.test.testng.NewTest"/>

      <!-- 第二个测试类：com.demo.test.testng.TestCase1 -->
      <class name="com.demo.test.testng.TestCase1"/>
    </classes>

  </test> <!-- 结束当前 Test 测试集 -->

</suite> <!-- 结束 Suite 测试套件 -->
```

执行结果解析（以「只执行 database 分组」为例）

运行 `testng.xml` 后，控制台输出如下（重点看分组钩子的触发时机）：

```
→ BeforeMethod 执行
test2 → 所属分组：group2 + database + smoke
→ AfterMethod 执行

→ BeforeMethod 执行
testDB → 所属分组：database + mysql + smoke
→ AfterMethod 执行

===== AfterGroups(database) 执行 =====
```

核心规律：

1. `@BeforeGroups(database)` 在 `database` 分组**第一个用例（test2）执行前**触发；
2. `@AfterGroups(database)` 在 `database` 分组**最后一个用例（testDB）执行后**触发；
3. 仅 `database` 分组的用例被执行，其他分组（如 group1、仅 smoke）的用例跳过；
4. 类级分组 `smoke` 是「隐式归属」，不影响 XML 对 `database` 分组的筛选。

关键用法对比表

分组场景	代码实现	执行效果
方法级单分组	`@Test(groups="group1")`	方法仅属于 group1（+ 类级分组）
方法级多分组	`@Test(groups={"a","b"})`	方法同时属于 a、b 两个分组（+ 类级分组）
类级分组	类上标注 `@Test(groups="smoke")`	所有方法默认继承 smoke 分组
分组专属钩子	`@BeforeGroups(groups="database")`	仅在 database 分组第一个用例执行前触发
XML 执行指定分组	`<include name="database"/>`	仅执行属于 database 分组的所有用例
XML 排除指定分组	`<exclude name="group1"/>`	执行除 group1 外的所有用例

常见问题排查

1. `@BeforeGroups` 不生效？→ 必须指定分组名（`groups="xxx"`），且该分组有执行的用例；
2. 类级分组不继承？→ 类上的 `@Test` 注解不能漏，方法级分组是「叠加」而非「覆盖」；
3. XML 配置后无用例执行？→ 检查分组名拼写（大小写敏感）、测试类全路径是否正确。

九， 分suite测试

1. 分 Suite 测试核心概念回顾

首先明确核心逻辑，帮你建立基础认知：

1. **测试套件（Suite）**：是测试用例 / 测试集的集合，TestNG 中通过 `testng.xml` 的 `<suite>` 标签定义（无法在代码中直接定义）；
2. **核心结构**：`suite`（根标签）→ 包含 1~N 个 `test`（测试集）→ 每个 `test` 包含 `groups`（分组）/ `classes`（测试类）；
3. **核心价值**：灵活配置要执行的测试（如按模块分 Suite、按环境分 Suite），支持多套件批量执行、并行执行等高级策略。

2. suite 标签的所有合法属性（附说明 + 示例）

`<suite>` 标签支持的全部核心属性，按「常用程度」排序

属性名	作用说明	合法取值	示例
`name`	定义套件名称（必填，用于标识套件）	自定义字符串（如 "Login_Suite"、"Payment_Suite"）	`<suite name="User_Module_Suite">`
`parallel`	定义套件内测试的并行执行策略	`false`（默认，串行）、`methods`（方法级并行）、`tests`（test 级并行）、`classes`（类级并行）、`instances`（实例级并行）	`<suite parallel="tests">`
`threadCount`	配合 `parallel` 使用，指定并行执行的线程数	正整数（如 2、5、10）	`<suite parallel="methods" threadCount="5">`
`verbose`	控制控制台输出日志的详细程度	0~10（数值越大，日志越详细，默认 1）	`<suite verbose="3">`
`preserve-order`	是否按 XML 中定义的顺序执行 `test`/`class`/`method`	`true`（默认，按顺序）、`false`（乱序）	`<suite preserve-order="true">`
`annotations`	指定注解解析器（极少用，默认 TestNG 自带）	`JDK`（默认）、`JAVADOC`	`<suite annotations="JDK">`
`time-out`	为套件内所有测试方法设置全局超时时间（毫秒）	正整数（如 5000 表示 5 秒超时）	`<suite time-out="5000">`
`skipfailedinvocationcounts`	是否跳过失败的调用计数统计	`true`、`false`（默认）	`<suite skipfailedinvocationcounts="false">`
`configfailurepolicy`	配置方法（如 @BeforeMethod）失败时，是否继续执行测试	`skip`（默认，跳过）、`continue`（继续）	`<suite configfailurepolicy="continue">`

3. 分 Suite 测试的实操案例

场景 1：单 Suite 包含多 Test（按模块拆分 Test）

适合按「功能模块」组织测试（如登录模块、支付模块），一个 Suite 包含多个 Test 集：

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- 总套件：用户模块测试 -->
<suite name="User_Module_Suite" parallel="false" verbose="2">
    <!-- Test1：登录模块测试集 -->
    <test name="Login_Test">
        <classes>
            <class name="com.demo.test.testng.LoginTest"/>
        </classes>
    </test>

    <!-- Test2：个人中心模块测试集 -->
    <test name="User_Center_Test">
        <groups>
            <include name="user_info"/>
        </groups>
        <classes>
            <class name="com.demo.test.testng.UserCenterTest"/>
        </classes>
    </test>

    <!-- Test3：退出登录测试集 -->
    <test name="Logout_Test">
        <classes>
            <class name="com.demo.test.testng.LogoutTest"/>
        </classes>
    </test>
</suite>
```

场景 2：多 Suite 配置（拆分多个 XML 文件，批量执行）

适合按「环境 / 阶段」拆分 Suite（如测试环境 Suite、生产环境 Suite），需创建 2 个 XML 文件：

① testng-dev.xml（测试环境套件）

```xml
<?xml version="1.0" encoding="UTF-8"?>
<suite name="Dev_Env_Suite" parallel="tests" threadCount="2">
    <test name="Dev_Login_Test">
        <classes>
            <class name="com.demo.test.testng.DevLoginTest"/>
        </classes>
    </test>
</suite>
```

② testng-prod.xml（生产环境套件）

```xml
<?xml version="1.0" encoding="UTF-8"?>
<suite name="Prod_Env_Suite" verbose="3">
    <test name="Prod_Payment_Test">
        <classes>
            <class name="com.demo.test.testng.ProdPaymentTest"/>
        </classes>
    </test>
</suite>
```

③ 批量执行多 Suite（创建总配置文件 testng-all.xml）

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- 批量执行多个 Suite 文件 -->
<suite name="All_Env_Suite">
    <suite-files>
        <suite-file path="testng-dev.xml"/>
        <suite-file path="testng-prod.xml"/>
    </suite-files>
</suite>
```

场景 3：Suite 级并行执行（提升测试效率）

适合用例较多的场景，按 `test` 级并行执行，节省时间：

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- 并行执行 Suite 内的所有 Test，线程数 3 -->
<suite name="Parallel_Suite" parallel="tests" threadCount="3">
    <test name="Test1">
        <classes><class name="com.demo.test.testng.TestCase1"/></classes>
    </test>
    <test name="Test2">
        <classes><class name="com.demo.test.testng.TestCase2"/></classes>
    </test>
    <test name="Test3">
        <classes><class name="com.demo.test.testng.TestCase3"/></classes>
    </test>
</suite>
```

4. 分 Suite 测试的核心最佳实践

1. **命名规范**：Suite 名称按「模块 / 环境 + Suite」命名（如 `Login_Dev_Suite`），Test 名称按「功能 + Test」命名（如 `WeChat_Pay_Test`），便于识别；

2. 并行策略选择：
    - 简单场景用 `parallel="false"`（串行）；
    - 用例独立无依赖时，用 `parallel="tests"`（Test 级并行）；
    - 方法无依赖时，用 `parallel="methods"`（方法级并行）；
3. **拆分原则**：按「功能模块」「测试环境」「优先级」拆分 Suite/Test，避免单个 XML 配置过于臃肿；
4. **全局配置**：通用属性（如 `verbose`、`time-out`）在 `<suite>` 标签定义，无需在每个 `<test>` 中重复配置。

总结

1. **核心结构**：TestNG 套件由 `<suite>` 定义（根标签），内部可包含 1~N 个 `<test>`，每个 `<test>` 对应一个测试集；
2. **核心属性**：`name`（必填）、`parallel` + `threadCount`（并行）、`verbose`（日志）是最常用的属性；
3. **核心价值**：分 Suite 测试支持灵活的测试配置（按模块 / 环境拆分、并行执行、批量运行），大幅提升测试管理效率。

十， 依赖测试

1. 依赖测试核心概念

TestNG 依赖测试用于**控制测试方法的执行顺序**，或实现「方法间数据 / 状态共享」，核心是通过注解声明测试方法之间的显式依赖关系，解决测试用例执行顺序混乱、数据依赖的问题。

2. 依赖测试的两种声明方式

TestNG 支持通过「方法名」或「分组名」声明依赖，两种方式均通过 `@Test` 注解的属性实现：

依赖方式	注解属性	作用说明
依赖指定方法	`dependsOnMethods`	声明当前测试方法依赖其他**具体测试方法**，需等待依赖方法执行完成后再执行
依赖指定分组	`dependsOnGroups`	声明当前测试方法依赖其他**测试分组**，需等待该分组所有方法执行完成后再执行

语法示例（基础）

```
// 1. 依赖指定方法：testB 依赖 testA，先执行 testA，再执行 testB
@Test
public void testA() {
    System.out.println("执行 testA");
}

@Test(dependsOnMethods = {"testA"})
public void testB() {
    System.out.println("执行 testB（依赖 testA）");
}

// 2. 依赖指定分组：testC 依赖 group1 分组，先执行 group1 所有方法，再执行 testC
@Test(groups = "group1")
public void testGroup1_1() {
```

```
    System.out.println("执行 group1 方法1");
}

@Test(dependsOnGroups = {"group1"})
public void testC() {
    System.out.println("执行 testC（依赖 group1 分组）");
}
```

3. 硬依赖 vs 软依赖（核心区别）

依赖测试分为「硬依赖」和「软依赖」，核心差异在于「依赖项失败时，当前方法是否执行」：

特性	硬依赖（默认）	软依赖（alwaysRun=true）
触发方式	默认生效，无需额外配置	需在 `@Test` 中添加 `alwaysRun=true`
依赖项失败时行为	当前方法**跳过执行**（报告中标为 skip）	当前方法**照常执行**（不受依赖项失败影响）
适用场景	依赖项是当前方法的「必要前提」（如登录→下单）	依赖项仅为「参考 / 数据补充」，无因果关系（如统计→报表展示）
语法示例	`@Test(dependsOnMethods = {"testA"})`	`@Test(dependsOnMethods = {"testA"}, alwaysRun = true)`

4. 完整实操案例（硬依赖 + 软依赖）

```
package com.demo.test.testng;

import org.testng.Assert;
import org.testng.annotations.Test;

public class DependencyTest {

    // ------------- 硬依赖示例 -------------
    // 被依赖方法：模拟执行失败
    @Test
    public void testLogin() {
        System.out.println("执行 testLogin（模拟登录失败）");
        Assert.fail("登录失败"); // 主动让方法失败
    }

    // 硬依赖 testLogin：因依赖方法失败，此方法会被跳过（skip）
    @Test(dependsOnMethods = {"testLogin"})
    public void testOrder() {
        System.out.println("执行 testOrder（依赖登录，硬依赖）");
    }

    // ------------- 软依赖示例 -------------
    // 被依赖方法：模拟执行失败
    @Test
    public void testDataStat() {
        System.out.println("执行 testDataStat（模拟数据统计失败）");
        Assert.fail("数据统计失败");
    }
```

```java
    // 软依赖 testDataStat：依赖方法失败，此方法仍会执行
    @Test(dependsOnMethods = {"testDataStat"}, alwaysRun = true)
    public void testReportShow() {
        System.out.println("执行 testReportShow（依赖统计，软依赖）");
        Assert.assertTrue(true); // 正常执行
    }

    // ------------- 依赖分组示例 -------------
    @Test(groups = "pay")
    public void testPay1() {
        System.out.println("执行 pay 分组方法1");
    }

    @Test(groups = "pay")
    public void testPay2() {
        System.out.println("执行 pay 分组方法2");
    }

    // 硬依赖 pay 分组：需等 pay 分组所有方法执行完才会执行
    @Test(dependsOnGroups = {"pay"})
    public void testPayResult() {
        System.out.println("执行 testPayResult（依赖 pay 分组）");
    }
}
```

执行结果解析

```
执行 testLogin（模拟登录失败）
// testLogin 失败，testOrder 被跳过（skip）
执行 testDataStat（模拟数据统计失败）
执行 testReportShow（依赖统计，软依赖） // 软依赖不受影响，正常执行
执行 pay 分组方法1
执行 pay 分组方法2
执行 testPayResult（依赖 pay 分组） // 分组依赖成功，正常执行
```

5. 关键注意事项

1. **依赖顺序优先级**：`dependsOnMethods` / `dependsOnGroups` 优先级高于 `priority`（执行优先级），先满足依赖，再按优先级执行；
2. **多依赖处理**：可依赖多个方法 / 分组（数组形式），如 `dependsOnMethods = {"testA", "testB"}`，需所有依赖项执行完才会执行当前方法；
3. **循环依赖禁止**：禁止声明循环依赖（如 testA 依赖 testB，testB 依赖 testA），会导致 TestNG 报错；
4. **软依赖使用原则**：仅当依赖项与当前方法「无因果关系」时使用，否则会导致当前方法无意义的失败（如依赖登录失败仍执行下单，必然失败）。
5. **被依赖方法fail**：会报错

十一，参数化测试

TestNG中的另一个有趣的功能是参数化测试。 在大多数情况下，您会遇到业务逻辑需要大量测试的场景。 参数化测试允许开发人员使用不同的值一次又一次地运行相同的测试。
TestNG可以通过两种不同的方式将参数直接传递给测试方法：

- 使用testng.xml
- 使用数据提供者

下面分别介绍两种传参方式：

1、使用textng.xml传送参数

范例代码如下：

```
public class TestCase1 {

    @Test(enabled=true)
    @Parameters({"param1", "param2"})
    public void TestNgLearn1(String param1, int param2) {
        System.out.println("this is TestNG test case1, and param1 is:"+param1+";
param2 is:"+param2);
        Assert.assertFalse(false);
    }

    @Test(dependsOnMethods= {"TestNgLearn1"})
    public void TestNgLearn2() {
        System.out.println("this is TestNG test case2");
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<suite name="Suite" parallel="false">
  <test name="Test">
    <parameter name="param1" value="1011111" />
    <parameter name="param2" value="10" />
    <classes>
      <class name="com.demo.test.testng.TestCase1"/>
    </classes>
  </test> <!-- Test -->
</suite> <!-- Suite -->
```

执行结果

```
this is TestNG test case1, and param1 is:1011111; param2 is:10
this is TestNG test case2

===============================================
Suite
Total tests run: 2, Failures: 0, Skips: 0
===============================================
```

2、使用@DataProvider传递参数

此处需要注意，传参的类型必须要一致，且带有@DataProvider注解的函数返回的必然是Object[][]，此处需要注意。
代码如下：

```
public class TestCase1 {

    @DataProvider(name = "provideNumbers")
    public Object[][] provideData() {

        return new Object[][] { { 10, 20 }, { 100, 110 }, { 200, 210 } };
```

```
    }

    @Test(dataProvider = "provideNumbers")
    public void TestNgLearn1(int param1, int param2) {
        System.out.println("this is TestNG test case1, and param1 is:"+param1+";
param2 is:"+param2);
        Assert.assertFalse(false);
    }

    @Test(dependsOnMethods= {"TestNgLearn1"})
    public void TestNgLearn2() {
        System.out.println("this is TestNG test case2");
    }
}
```

运行此class，结果为：

```
this is TestNG test case1, and param1 is:10; param2 is:20
this is TestNG test case1, and param1 is:100; param2 is:110
this is TestNG test case1, and param1 is:200; param2 is:210
this is TestNG test case2
PASSED: TestNgLearn1(10, 20)
PASSED: TestNgLearn1(100, 110)
PASSED: TestNgLearn1(200, 210)
PASSED: TestNgLearn2
```

十二，XML配置文件说明

前面讲的大多都是以测试脚本为基础来运行的，少部分是以xml运行，这里以xml来讲解下：

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd" >
<suite name="SuiteName" verbose="1" >
```

1、suite标签

testNG.xml文件的最外层标签即suite，即测试套件，其下可以有多个和，其有几个可以添加的属性在第十节的分suite测试中有做说明，这里做下详细说明：

(1)、name属性
此属性属于必须要有的，值可以自行设定，此名字会在testNG的报告中看到；

(2)、verbose属性
此属性为指定testNG报告的详细程度，从0开始到10，其中10为最详细，默认生成的xml此属性值为1；

(3)、parallel属性
此属性是指代运行方式，默认为none，即串行运行方式；并行执行方法包括如下几种，下面做分别说明：

- methods：方法层级，若为此值，则该suite下所有的测试方法都将进行多线程，即测试用例级别的多线程。如果用例之间有依赖，则执行顺序会按照设定的依赖来运行；
- ```
  <suite name="My suite" parallel="methods" thread-count="5">
  ```
  ```

- tests：TestNG将在同一线程中运行相同的标签中的所有方法，每个标签都将处于一个单独的线程中，这允许您将不是线程安全的所有类分组在同一个中，并保证它们都将在同一个线程中运行，同时利用TestNG使用尽可能多的线程运行测试。
- ```
  <suite name="My suite" parallel="tests" thread-count="5">
  ```
- classes：类级别并发，即TestNG会将该suite下每个class都将在单独的线程中运行，同一个class下的所有用例都将在同一个线程中运行；
- ```
  <suite name="My suite" parallel="classes" thread-count="5">
  ```
- instances：实例级别，即TestNG将在同一线程中运行同一实例中的所有方法，两个不同实例上的两个方法将在不同的线程中运行。
- ```
  <suite name="My suite" parallel="instances" thread-count="5">
  ```

(4)、thread-count属性
此属性用于指定线程数，按照需要输入，需要parallel参数非none时才可以添加；

(5)、annotations属性
此项为注解的级别，为methods级别和class级别，一般不用设置；

(6)、time-out属性
此属性用于指定超时时间，该suite下所有的用例的超时时间；

(7)、group-by-instances属性
此项用于那些有依赖的方法，且被依赖的对象有多个重载对象，因为如果是依赖方法，且该方法有多个重载方法，则默认是会将所有重载方法都跑完再运行被依赖方法，但有时候我们不想这样，则将此项设置为true即可；

(8)、preserve-order属性
值可输入true或者false，如果为true，则用例执行会按照在xml中的顺序执行，否则会乱序执行，不添加此属性的话默认是按顺序执行的；

2、test标签

此标签无特别意义，其下可以包括多个标签，如groups、classes等，如下介绍下几种书写方式：

1）选择一个包中的全部测试脚本（包含子包）

```
<test name = "allTestsInAPackage" >
   <packages>
      <package name = "whole.path.to.package.* />
   </packages>
</test>
```

2）选择一个类中的全部测试脚本

```
<test name = "allTestsInAPackage" >
   <packages>
      <package name = "whole.path.to.package.* />
   </packages>
</test>
```

（3）选择一个类中的部分测试脚本
```

```
<test name = "aFewTestsFromAClass" >
   <classes>
  <class name="whole.path.to.package.className >
      <methods>
         <include name = "firstMethod" />
         <include name = "secondMethod" />
         <include name = "thirdMethod" />
      </methods>
  </class>
   </classes>
</test>
```

（4）选择一个包中的某些组

```
<test name = "includedGroupsInAPackage" >
   <groups>
      <run>
         <include name = "includedGroup" />
      </run>
   </groups>
   <packages>
      <package name = "whole.path.to.package.* />
   </packages>
</test>
```

（5）排除一个包中的某些组

```
<test name = "excludedGroupsInAPackage" >
   <groups>
      <run>
         <exclude name = "excludedGroup" />
      </run>
   </groups>
   <packages>
      <package name = "whole.path.to.package.* />
   </packages>
</test>
```

可附带的属性

附带的属性有如下几种，下面对各个属性做单独说明：

(1)、name属性
此属性属于必须要有的，值可以自行设定，此名字会在testNG的报告中看到；

(2)、verbose属性
此属性为指定testNG报告的详细程度，从0开始到10，其中10为最详细，默认生成的xml此属性值为1

(3)、threadPoolSize属性
该属性指定此test的线程池大小，为数字；

```
<test name = "excludedGroupsInAPackage" >
   <groups>
      <run>
         <exclude name = "excludedGroup" />
      </run>
   </groups>
   <packages>
      <package name = "whole.path.to.package.* />
   </packages>
</test>
```

(4)、invocationCount属性
该属性指定此test的运行次数，为数字，范例如上面的代码所示；

(5)、time-out属性
此属性用于指定超时时间，该suite下所有的用例的超时时间，范例如上面的代码所示；

(6)、group-by-instances属性
此项用于那些有依赖的方法，且被依赖的对象有多个重载对象，因为如果是依赖方法，且该方法有多个重载方法，则默认是会将所有重载方法都跑完再运行被依赖方法，但有时候我们不想这样，则将此项设置为true即可；

```
<suite name="Factory" group-by-instances="true">
```

(7)、preserve-order属性
值可输入true或者false，如果为true，则用例执行会按照在xml中的顺序执行，否则会乱序执行，不添加此属性的话默认是按顺序执行的；

3、group标签

此标签必然是在标签下的，用于标识那些组会被用于测试或者被排除在测试之外，其同级必然要包含一个标签或者标签，用于指定groups来自于哪些包或者类；
如下即为包含一个group，排除一个group的例子：

```
<groups>
  <run>
     <include name = "includedGroupName" />
     <exclude name = "excludedGroupName" />
  </run>
</groups>
```

高级应用：

```
<test name="Regression1">
  <groups>
    <define name="functest">
      <include name="windows"/>
      <include name="linux"/>
    </define>

    <define name="all">
      <include name="functest"/>
      <include name="checkintest"/>
    </define>
```

```
    <run>
      <include name="all"/>
    </run>
  </groups>

  <classes>
    <class name="test.sample.Test1"/>
  </classes>
</test>
```

4、其他

其他的话就是测试脚本的选择了，有三种方式：

（1）选择一个包

```
<packages>
    <package name = "packageName" />
</packages>
```

（2）选择一个类

```
<classes>
    <class name = "className" />
</classes>
```

（3）选择一个方法

```
<classes>
    <class name = "className" />
       <methods>
          <include name = "methodName" />
       </methods>
    </class>
</classes>
```

这里也支持正则表达式，例如：

```
<test name="Test1">
  <classes>
    <class name="example1.Test1">
      <methods>
        <include name=".*enabledTestMethod.*"/>
        <exclude name=".*brokenTestMethod.*"/>
      </methods>
     </class>
  </classes>
</test>
```

十三，TestNG报告

默认报告输出位置为当前工程的test-output文件夹下，包括xml格式和html格式。
如果想要美化报告，则按照如下步骤：

1、配置：Eclipse --> Window --> Preferences -->testng

2、勾选Disable default listeners

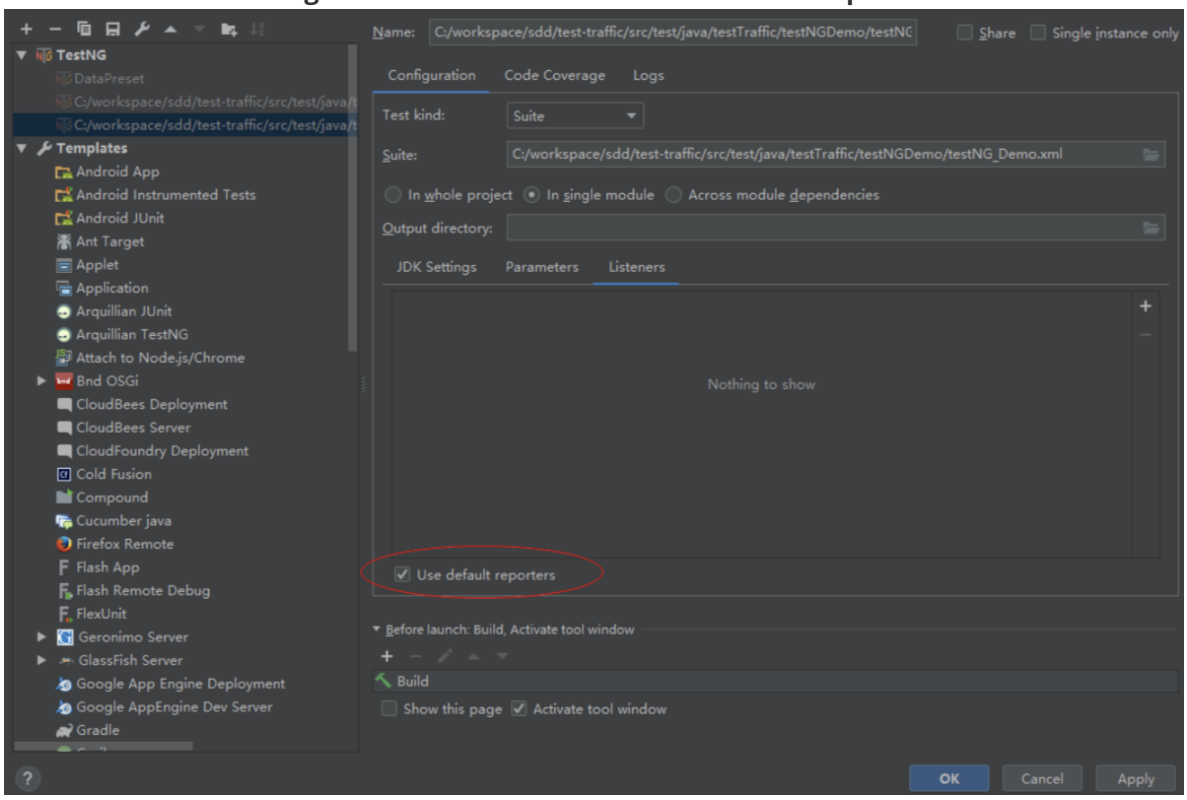
3、在Pre Defined Listeners 输入框中输入org.uncommons.reportng.HTMLReporter记得在POM上添加如下代码：

```
<dependency>
      <groupId>org.uncommons</groupId>
      <artifactId>reportng</artifactId>
      <version>1.1.4</version>
      <scope>test</scope>
      <exclusions>
        <exclusion>
          <groupId>org.testng</groupId>
          <artifactId>testng</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
    <dependency>
      <groupId>com.google.inject</groupId>
      <artifactId>guice</artifactId>
      <version>3.0</version>
      <scope>test</scope>
    </dependency>
```

不然无法运行的。
如上图所示，还可以自定义testng.xml的模板，并在上图中指定。

使用IDEA+TestNG进行测试，没有生成 测试报告，是因为没有勾选监听器使用默认报告，具体操作如下：

“Run” -> "Edit Configurations" -> "listeners" -> "Use default reporters"



输出报告在test-output文件夹中（index.html）

十四，与Junit的区别

功能	实现	
	JUnit 4	TestNG
测试注释	@Test	@Test
套件测试	@RunWith和@Suite	<suite>标签
在套件中的所有测试运行之前运行	-	@BeforeSuite
在套件中的所有测试运行之后运行	-	@AfterSuite
测试之前运行	-	@BeforeTest
测试之后运行	-	@AfterTest
分组测试	-	@Test的group参数
在调用属于任何这些组的第一个测试方法之前运行	-	@BeforeGroups
在调用属于任何这些组的第一个测试方法之后运行	-	@AfterGroups
在调用当前类的第一个测试方法之前运行	@BeforeClass	@BeforeClass
在调用当前类的第一个测试方法之后运行	@AfterClass	@AfterClass
在每个测试方法之前运行	@Before	@BeforeMethod
在每个测试方法之后运行	@After	@AfterMethod
忽略测试	@ignore	@Test(enbale=false)
预期的异常	@Test(expected = ArithmeticException.class)	@Test(expectedExceptions = ArithmeticException.class)
超时测试	@Test(timeout = 1000)	@Test(timeout = 1000)
参数化测试	@RunWith和@Parameter	@Parameters或@DataProvider
依赖测试	-	@Test的dependsOnMethods参数

http://blog.csdn.net/lovedingd